Índice

CONSIGNA – SPRINT 0: INICIO DEL PROYECTO **2**
1. Propósito de aprendizaje **2**
2. Entregables: **2**
a. Documento de selección del enfoque del proyecto (con justificación técnica) 2
b. Declaración de la visión del proyecto 3
c. Project Charter 3
1. Título del Proyecto 4
2. Carta del Proyecto 4
3. Detalles del Proyecto 4
4. Lista de interesados (Stakeholders) 5
5. Resumen del cronograma de hitos 5
6. Consideraciones del proyecto 6
d. Registro de supuestos y restricciones 7
Supuestos 7
Restricciones 7
a. Declaración del equipo del proyecto 8
a. Repositorio GitHub operativo 9
b. Documento inicial del problema (primer borrador) 9
1. Contexto del problema 9
2. Problema central 10
Ambigüedad del problema 10
3. Variables involucradas 10
4. Restricciones del problema 11
5. Necesidad de una solución tecnológica 11
6. Alcance del proyecto 11
c. Lista preliminar de requerimientos funcionales y no funcionales 11
1. Requerimientos funcionales 11
2. Requerimientos no funcionales 13
● **Criterios de desempeño:** **14**

Proyecto de Fin de Asignatura (PFA)

CONSIGNA – SPRINT 0: INICIO DEL PROYECTO

1. Propósito de aprendizaje

El estudiante será capaz de analizar y estructurar un problema complejo de ingeniería de software, utilizando Scrum, GitHub y técnicas de levantamiento de requerimientos, aplicando principios de análisis de sistemas, gestión de proyectos y pensamiento crítico, de manera rigurosa, colaborativa y responsable, con el objetivo de definir una base sólida y justificable para el desarrollo de una aplicación web moderna.

En este Sprint 0, el equipo deberá comprender el problema propuesto en el contexto del desarrollo de una aplicación web moderna, caracterizado por múltiples variables, restricciones, ambigüedad en los requerimientos y la necesidad de toma de decisiones fundamentadas. El trabajo se desarrollará bajo un enfoque ágil, promoviendo la organización efectiva del equipo y la definición inicial del proyecto.

El equipo deberá realizar un análisis detallado de la consigna del proyecto, identificando el problema central, sus componentes y limitaciones. A partir de ello, elaborará una primera aproximación de los requerimientos funcionales y no funcionales, reconociendo posibles ambigüedades, inconsistencias o vacíos de información. Asimismo, deberá seleccionar un enfoque de desarrollo (tecnológico y metodológico), justificando cada decisión en función del contexto, restricciones y objetivos del proyecto.

Como parte de la formalización del inicio del proyecto, el equipo elaborará los siguientes documentos: declaración de la visión del proyecto, Project Charter, registro de supuestos y restricciones, y declaración del equipo del proyecto (roles, responsabilidades y normas de trabajo). Además, deberá crear un repositorio en GitHub, configurando una estructura básica del proyecto y asegurando su correcto acceso y uso colaborativo.

2. Entregables:

a. Documento de selección del enfoque del proyecto (con justificación técnica)

Criterio	**MERN (MongoDB, Express, React, Node.js)**	**Django (Python)**
Lenguaje principal	JavaScript en todo el stack	Python en backend y JavaScript en frontend
Desarrollo frontend	React permite crear interfaces dinámicas y modernas	Django usa plantillas o requiere frameworks adicionales

Desarrollo backend	Node.js y Express facilitan la creación de APIs REST	Django ofrece un framework completo pero más estructurado
Base de datos	MongoDB permite almacenar datos de forma flexible	PostgreSQL o SQLite con esquema relacional rígido
Integración frontend-backend	Alta integración al usar JavaScript en todo el sistema	Requiere integración entre diferentes tecnologías
Curva de aprendizaje	Más sencilla si el equipo conoce JavaScript	Puede ser más compleja si no se domina Python

b. Declaración de la visión del proyecto

Desarrollar una aplicación web moderna que permita gestionar y generar horarios universitarios de manera eficiente, facilitando la planificación académica y evitando conflictos entre cursos, docentes y aulas, mediante una interfaz intuitiva y accesible para los usuarios del sistema.

c. Project Charter

1. Título del Proyecto

SISTEMA DE GENERACIÓN ÓPTIMA DE HORARIOS ACADÉMICOS EN ENTORNOS DE CURRÍCULO FLEXIBLE

2. Carta del Proyecto

El presente proyecto tiene como finalidad desarrollar una solución que permita generar horarios académicos óptimos en entornos universitarios con currículo flexible. Este tipo de entorno presenta alta complejidad debido a la variabilidad en la matrícula, restricciones académicas y disponibilidad de recursos como docentes y aulas.

El sistema propuesto busca automatizar el proceso de asignación de horarios considerando múltiples restricciones y condiciones, permitiendo obtener soluciones válidas que eviten conflictos y optimicen la distribución de recursos.

Con ello, se espera mejorar la eficiencia en la planificación académica, reducir errores manuales y facilitar la toma de decisiones en instituciones educativas.

3. Detalles del Proyecto

3.1. Descripción del producto / Entregables

Descripción del producto

El proyecto consiste en el desarrollo de un sistema que permita generar horarios académicos de forma automática, considerando múltiples restricciones como disponibilidad de docentes, aulas, cursos y estudiantes.

El sistema ayudará a reducir conflictos, optimizar recursos y facilitar la planificación académica en universidades con currículo flexible.

Entregables

- Documento de análisis del problema
- Modelo del problema (restricciones y variables)
- Diseño de la solución
- Sistema funcional de generación de horarios
- Pruebas del sistema
- Informe final del proyecto
- Video demostrativo

3.2. El proyecto NO incluye

- Implementación en sistemas reales de universidades
- Integración con sistemas institucionales existentes
- Infraestructura adicional
- Capacitación formal a usuarios finales

- Soporte posterior a la entrega del proyecto

3.3. Recursos pre-asignados

- Equipo de estudiantes de ingeniería de sistemas
- Computadoras personales
- Acceso a herramientas de desarrollo
- Tiempo asignado para desarrollo del proyecto
- Conocimiento básico del dominio académico

4. Lista de interesados (Stakeholders)

Nombre / Grupo	Rol	Responsabilidad / Interés
Equipo de desarrollo (estudiantes)	Desarrolladores del sistema	Analizar el problema, diseñar la solución, desarrollar el sistema y realizar pruebas del software.
Docente del curso	Supervisor del proyecto	Guiar el desarrollo del proyecto, evaluar el progreso del equipo y validar que se cumplan los objetivos del curso.
Coordinador académico	Usuario principal del sistema	Utilizar el sistema para generar y gestionar horarios académicos de manera eficiente.
Docentes universitarios	Usuarios del sistema	Consultar los horarios asignados y registrar su disponibilidad horaria.
Estudiantes universitarios	Usuarios finales	Consultar los horarios generados para conocer sus clases y organización académica.
Universidad / Institución educativa	Beneficiario del sistema	Mejorar la planificación académica y optimizar el uso de recursos como aulas y docentes.

5. Resumen del cronograma de hitos

Hito	Descripción	Fecha
Hito 1	Análisis del problema y requerimientos	Semana 1 – Semana 2
Hito 2	Diseño de la solución	Semana 3 – Semana 4
Hito 3	Desarrollo del sistema	Semana 5 – Semana 9

Hito 4	Pruebas y validación	Semana 10 – Semana 11
Hito 5	Entrega final del proyecto	Semana 12

6. Consideraciones del proyecto

6.1. Riesgos de alto nivel

- Complejidad del problema debido a múltiples restricciones
- Cambios en los requerimientos durante el desarrollo
- Limitaciones en tiempo y recursos
- Dificultad para obtener soluciones óptimas
- Falta de experiencia en modelado del problema

6.2. Criterios de aceptación

- El sistema genera horarios sin conflictos
- Se cumplen las restricciones principales del problema
- Los horarios son válidos y utilizables
- El sistema funciona correctamente en pruebas
- Se entrega toda la documentación requerida

6.3. Suposiciones

- Se contará con información suficiente para modelar el problema
- Los datos de entrada serán válidos
- El equipo tendrá disponibilidad para desarrollar el proyecto
- Las restricciones del problema se mantendrán estables

6.4. Restricciones

- Tiempo limitado para el desarrollo
- Recursos técnicos limitados
- Proyecto desarrollado solo por estudiantes
- No se contará con datos reales completos
- Alcance limitado a un prototipo funcional

d. Registro de supuestos y restricciones

Supuestos

Los supuestos representan condiciones que se consideran verdaderas para el desarrollo del proyecto, aunque no estén completamente confirmadas.

- La universidad cuenta con docentes disponibles para registrar en el sistema.
- Cada curso tiene un docente asignado.
- Las aulas tienen capacidad suficiente para los cursos.
- Los usuarios del sistema tienen conocimientos básicos de uso de aplicaciones web.
- La universidad dispone de una estructura académica organizada por cursos, docentes y aulas.
- Los coordinadores académicos son responsables de la planificación y organización de los horarios.
- Los docentes tienen horarios de disponibilidad previamente definidos.
- Cada aula puede ser utilizada por un solo curso en un horario específico.
- Los cursos tienen una duración determinada dentro del horario académico.
- Los usuarios del sistema tendrán acceso a internet para utilizar la aplicación web.
- Los integrantes del equipo cuentan con conocimientos básicos de programación web.
- El equipo tiene acceso a herramientas de desarrollo como GitHub y entornos de programación.

Restricciones

Las restricciones son limitaciones o condiciones que deben respetarse durante el desarrollo del proyecto.

- El sistema debe desarrollarse utilizando el stack tecnológico MERN (MongoDB, Express, React y Node.js).
- El proyecto debe completarse dentro del periodo académico establecido de 12 semanas.
- El desarrollo se realizará en equipo de trabajo, por lo que se requiere coordinación entre los integrantes.
- El sistema será desarrollado como un Producto Mínimo Viable (PMV), por lo que solo se implementarán las funcionalidades esenciales.
- El proyecto debe gestionarse utilizando metodologías ágiles, específicamente Scrum.
- El código fuente del proyecto deberá almacenarse y gestionarse en un

repositorio GitHub.

- Se deberá utilizar Git como sistema de control de versiones.
- El repositorio deberá evidenciar el trabajo colaborativo mediante commits, ramas y pull requests.
- El sistema deberá dividirse en frontend y backend, respetando una arquitectura organizada.
- La documentación del proyecto deberá elaborarse en formato Markdown dentro del repositorio.
- El sistema deberá ser accesible desde navegadores web modernos.
- El proyecto deberá cumplir con los lineamientos establecidos en la consigna del curso.

a. Declaración del equipo del proyecto

Datos del Equipo
Nombre del Equipo:
Líder de Equipo: Vasquez Miranda Luis Alexis
Miembros del Equipo:

- Araujo Huamani, Leonardo Daniel
- Curi Untiveros Jefferson Diego
- Vasquez Miranda Luis Alexis
- Vilcarano De La Cruz Frank

Plan de Reuniones
Lugar de reunión: Reuniones meet
Frecuencia: Una vez a la semana
Día: Domingo Horario: 4:00-6:00 p.m.

Roles y Responsabilidades

Nombre	Título	Rol / Responsabilidad
Araujo Huamani Leonardo Daniel	Analista de Sistemas	Analizar el problema del sistema de generación de horarios, identificar variables, restricciones y requerimientos del sistema. Participar en la definición de funcionalidades y validar que el sistema responda al problema planteado.
Curi Untiveros Jefferson Diego	Diseñador de Software	Diseñar la arquitectura del sistema, definir la estructura del frontend, backend y base de datos. Elaborar el diseño técnico de la solución y apoyar en la planificación del desarrollo del sistema.
Vasquez Miranda Luis Alexis	Líder del Proyecto / Desarrollador	Coordinar el trabajo del equipo, organizar las tareas del proyecto y supervisar el avance del desarrollo. Implementar funcionalidades del backend y asegurar la

	Backend	integración entre los componentes del sistema.
Vilcarano De La Cruz Frank Anthony	Desarrollador Frontend y QA	Desarrollar la interfaz del sistema, implementar las funcionalidades del frontend y apoyar en la integración con el backend. Realizar pruebas del sistema para verificar su funcionamiento y detectar posibles errores.

Objetivo del Equipo
El equipo tiene como objetivo desarrollar un sistema funcional que permita generar horarios académicos óptimos, cumpliendo con los requerimientos establecidos y trabajando de manera colaborativa.

Normas de Trabajo

- Cumplir con fechas de entrega
- Participar activamente en reuniones
- Comunicar avances y problemas
- Usar herramientas compartidas

Firma del Equipo

Araujo Huamani Leonardo Daniel	Curi Untiveros Jefferson Diego
Vasquez Miranda Luis Alexis	Vilcarano De La Cruz Frank

a. **Repositorio GitHub operativo**

https://github.com/NawaCruz/Horarios-acad-micos.git

b. **Documento inicial del problema (primer borrador)**

1. Contexto del problema

En las universidades, la planificación de horarios académicos es una tarea compleja que involucra múltiples variables como la disponibilidad de docentes, la asignación de aulas, los cursos ofrecidos, las secciones disponibles y las restricciones académicas asociadas. Generalmente, este proceso se realiza de forma manual o mediante herramientas poco automatizadas, lo que puede generar errores, conflictos de horarios y dificultades en la organización académica.

Además, la coordinación de estos elementos requiere considerar diversas restricciones,

como la capacidad de las aulas, la disponibilidad horaria de los docentes, los prerrequisitos entre cursos y la distribución adecuada de las cargas académicas. La ausencia de herramientas adecuadas para gestionar estos factores puede provocar solapamientos de horarios, uso ineficiente de recursos y retrasos en la planificación académica.

2. Problema central

El problema central consiste en la dificultad de generar horarios académicos universitarios de manera eficiente, considerando múltiples restricciones y recursos limitados. La generación manual de horarios suele ser propensa a errores, conflictos entre cursos, docentes o aulas, y puede requerir múltiples ajustes antes de obtener una solución válida.

Esta situación genera pérdida de tiempo en los coordinadores académicos y puede afectar la correcta planificación de las actividades educativas.

Ambigüedad del problema

- No se especifica el número exacto de bloques horarios disponibles en el sistema.
- No se define la prioridad entre las diferentes restricciones cuando ocurre un conflicto.
- No se detalla si los docentes pueden dictar múltiples secciones del mismo curso en diferentes horarios.
- No se establece si los estudiantes pueden elegir libremente sus cursos o si siguen un plan de estudios predefinido.
- No se define el número máximo de cursos que un estudiante puede llevar por periodo académico.

3. Variables involucradas

El problema de generación de horarios involucra diferentes variables que deben ser consideradas para construir una solución válida. Entre las principales variables se encuentran:

- Estudiantes matriculados en los cursos.
- Docentes responsables de cada asignatura.
- Cursos y secciones disponibles en el periodo académico.
- Aulas disponibles y su capacidad.
- Bloques horarios definidos para el funcionamiento académico.
- Restricciones asociadas a disponibilidad y compatibilidad.

Estas variables deben combinarse adecuadamente para evitar conflictos y garantizar

una planificación académica funcional.

4. Restricciones del problema

El proceso de generación de horarios debe respetar diversas restricciones, entre ellas:

- Un docente no puede estar asignado a dos cursos en el mismo horario.
- Un aula no puede ser utilizada por más de un curso simultáneamente.
- Los estudiantes no deben tener cursos obligatorios en el mismo horario.
- La capacidad del aula debe ser suficiente para el número de estudiantes.
- Deben respetarse los prerrequisitos entre cursos.

Estas restricciones hacen que la generación de horarios sea un problema complejo que requiere un enfoque sistemático para su resolución.

5. Necesidad de una solución tecnológica

Debido a la complejidad del problema, resulta necesario desarrollar una herramienta tecnológica que permita automatizar el proceso de generación de horarios académicos. Una aplicación web moderna puede facilitar la gestión de datos académicos, la validación de restricciones y la generación automática de soluciones válidas.

El desarrollo de esta herramienta permitirá mejorar la eficiencia en la planificación académica, reducir errores humanos y optimizar el uso de recursos institucionales.

6. Alcance del proyecto

Como parte del proyecto del curso, se desarrollará un **Producto Mínimo Viable (PMV)** de una aplicación web para la generación de horarios universitarios. Este sistema permitirá registrar la información básica del problema, aplicar restricciones académicas y generar horarios de forma automática.

El sistema será desarrollado utilizando el **stack tecnológico MERN (MongoDB, Express, React y Node.js)** y se enfocará en demostrar la viabilidad de una solución tecnológica para el problema planteado.

c. Lista preliminar de requerimientos funcionales y no funcionales

1. Requerimientos funcionales

Los requerimientos funcionales describen lo que el sistema debe hacer.

Código	Requerimiento funcional	Descripción
RF01	Registro de estudiantes	El sistema debe permitir registrar estudiantes con sus datos básicos y su información académica necesaria para la generación de horarios.
RF02	Registro de docentes	El sistema debe permitir registrar docentes junto con su disponibilidad horaria, cursos asignables y restricciones asociadas.
RF03	Registro de cursos	El sistema debe permitir registrar cursos, indicando créditos, prerrequisitos, sección y demás datos necesarios para la planificación.
RF04	Registro de aulas	El sistema debe permitir registrar aulas con su capacidad, tipo y disponibilidad.
RF05	Gestión de disponibilidad horaria	El sistema debe permitir definir y actualizar bloques horarios disponibles para docentes, aulas y cursos.
RF06	Validación de prerrequisitos	El sistema debe verificar que los estudiantes cumplan con los prerrequisitos exigidos antes de considerar su matrícula en un curso.
RF07	Validación de límite de créditos	El sistema debe validar que la carga académica del estudiante se mantenga dentro del rango permitido por las reglas definidas para el problema.
RF08	Generación automática de horarios	El sistema debe generar horarios académicos automáticamente considerando restricciones de estudiantes, docentes, cursos y aulas.
RF09	Detección de conflictos	El sistema debe identificar conflictos de horario, como solapamientos de aulas, docentes o estudiantes.
RF10	Aplicación de restricciones académicas	El sistema debe considerar restricciones como no solapamiento, disponibilidad, prerrequisitos y compatibilidad de recursos durante la generación del horario.
RF11	Visualización de horarios generados	El sistema debe mostrar los horarios generados de manera clara para estudiantes, docentes y coordinadores.
RF12	Consulta de horarios por actor	El sistema debe permitir consultar horarios por estudiante, docente, curso, aula o sección.
RF13	Edición o ajuste manual de horarios	El sistema debe permitir realizar ajustes manuales sobre horarios generados cuando sea necesario.
RF14	Re-generación de horarios	El sistema debe permitir generar nuevamente el horario cuando cambien restricciones, disponibilidad o datos de entrada.
RF15	Priorización de restricciones	El sistema debe permitir definir restricciones obligatorias y restricciones deseables para mejorar la calidad de la solución.
RF16	Registro de restricciones del problema	El sistema debe permitir ingresar y mantener restricciones asociadas al modelo de horarios.
RF17	Generación de solución válida	El sistema debe asegurar que el resultado final cumpla con las restricciones obligatorias del problema.
RF18	Gestión de usuarios del sistema	El sistema debe permitir gestionar perfiles básicos como administrador, coordinador académico, docente y estudiante.

RF19	Reporte de conflictos o incidencias	El sistema debe generar reportes cuando no sea posible construir un horario completamente válido o cuando existan restricciones incompatibles.
RF20	Historial de ejecuciones	El sistema debe guardar un registro de las ejecuciones de generación de horarios para su revisión posterior.

2. Requerimientos no funcionales

Los requerimientos no funcionales describen cómo debe comportarse el sistema y sus atributos de calidad. La consigna menciona especialmente rendimiento, escalabilidad, usabilidad, seguridad y mantenibilidad bajo ISO/IEC 25010.

Código	Requerimiento no funcional	Descripción
RNF01	Rendimiento	El sistema debe generar horarios en un tiempo razonable para el volumen de datos definido en las pruebas del proyecto.
RNF02	Escalabilidad	El sistema debe poder manejar incrementos en la cantidad de estudiantes, cursos, docentes y aulas sin degradación crítica del servicio.
RNF03	Usabilidad	La interfaz debe ser clara, comprensible y fácil de utilizar por usuarios no técnicos.
RNF04	Seguridad	El sistema debe proteger la información registrada mediante control de acceso y mecanismos básicos de seguridad.
RNF05	Mantenibilidad	El sistema debe desarrollarse de forma modular y documentada para facilitar correcciones y mejoras futuras.
RNF06	Disponibilidad	El sistema debe estar disponible durante el periodo de uso académico previsto para pruebas y demostración.
RNF07	Integridad de datos	El sistema debe garantizar que la información almacenada no se corrompa ni presente inconsistencias lógicas.
RNF08	Compatibilidad web	El sistema debe funcionar correctamente en navegadores web modernos.
RNF09	Accesibilidad	La interfaz debe considerar principios básicos de accesibilidad para facilitar el uso por distintos perfiles de usuario, en línea con WCAG mencionada en la consigna.

RNF10	Calidad del código	El sistema debe seguir buenas prácticas de programación, con código limpio, modular y entendible, tal como exige la rúbrica.
RNF11	Portabilidad	El sistema debe poder ejecutarse en distintos equipos de desarrollo sin cambios mayores en su configuración.
RNF12	Trazabilidad	El sistema debe permitir rastrear cambios importantes en datos de horarios, restricciones y ejecuciones.
RNF13	Confiabilidad	El sistema debe producir resultados consistentes al usar los mismos datos de entrada y restricciones.
RNF14	Eficiencia en uso de recursos	El sistema debe utilizar de manera razonable la memoria y el procesamiento disponible, considerando las limitaciones técnicas planteadas en la consigna.
RNF15	Sostenibilidad	El sistema debe procurar un uso eficiente de recursos computacionales, alineado con el enfoque de Green Software mencionado en el documento.

- **Criterios de desempeño:**
  a. Claridad y profundidad en el análisis del problema, evidenciando identificación de ambigüedades y restricciones
  b. Coherencia y solidez en la justificación del enfoque seleccionado
  c. Calidad, estructura y formalidad de los documentos elaborados
  d. Uso adecuado de herramientas (GitHub, documentación)
  e. Nivel de organización, distribución de roles y trabajo colaborativo del equipo

El equipo deberá evidenciar explícitamente la naturaleza de problema complejo, identificando supuestos, reconociendo restricciones reales y argumentando las decisiones iniciales adoptadas.

RÚBRICA DE EVALUACIÓN - SPRINT 0: INICIO DEL PROYECTO

Criterio/Indicador	Sobresaliente (3)	Suficiente (2)	En desarrollo (1)	Insatisfactorio (0)
Análisis del problema: identificación clara del problema central, variables, stakeholders, ambigüedades y restricciones; coherencia del documento inicial	Análisis completo, identifica ≥4 ambigüedades y ≥4 restricciones reales, incluye stakeholders y relaciones; documento estructurado y coherente, aporta mejoras	Identifica problema y elementos básicos, ≥2 ambigüedades y ≥2 restricciones; documento claro sin errores críticos	Identificación parcial del problema, <2 ambigüedades o restricciones; documento incompleto o poco claro	No identifica correctamente el problema, sin ambigüedades ni restricciones; documento ausente o incorrecto
Requerimientos: definición de requerimientos funcionales y no funcionales, claridad, trazabilidad y coherencia	≥8 RF y ≥5 RNF bien definidos, claros, verificables y trazables al problema; sin inconsistencias	≥5 RF y ≥3 RNF adecuados, comprensibles, con mínima trazabilidad	RF/RNF incompletos, ambiguos o con inconsistencias; <5 RF o <3 RNF	No presenta requerimientos o son incorrectos
Selección del enfoque: justificación técnica del stack tecnológico y metodología (Scrum), coherencia con el problema y restricciones	Justificación sólida, compara ≥2 alternativas, argumenta decisiones con criterios técnicos y restricciones	Justificación básica del enfoque, sin comparación profunda pero coherente	Justificación débil, poco alineada al problema o sin sustento técnico	No justifica o la selección es incoherente
Declaración de la visión: claridad, alineación con el problema, definición de valor y alcance	Visión clara, concisa, alineada al problema, define valor y alcance medible; aporta mejoras	Visión adecuada, alineada, sin errores críticos	Visión ambigua o incompleta, con debilidades en valor o alcance	No presenta o es incorrecta
Project Charter: inclusión de objetivos, alcance, stakeholders, entregables, riesgos iniciales y cronograma preliminar	Documento completo con todos los elementos, bien estructurado, coherente y detallado	Incluye la mayoría de elementos, estructura adecuada sin errores críticos	Faltan varios elementos o presenta inconsistencias	No presenta o es incorrecto
Supuestos y restricciones: identificación, justificación y	≥5 supuestos y ≥5 restricciones justificadas y coherentes con el	≥3 supuestos y ≥3 restricciones identificadas	<3 supuestos o restricciones, o sin	No presenta o es incorrecto

Criterio/Indicador	Sobresaliente (3)	Suficiente (2)	En desarrollo (1)	Insatisfactorio (0)
coherencia con el problema	análisis	sin errores críticos	justificación clara	
Equipo de proyecto: definición de roles, responsabilidades, normas de trabajo y organización colaborativa	Roles claros (≥4), responsabilidades definidas, normas explícitas y coherentes con Scrum	Roles y responsabilidades básicos definidos, sin errores críticos	Roles poco claros o incompletos, sin normas definidas	No presenta o es incorrecto
Repositorio GitHub: creación, estructura inicial, control de versiones y acceso colaborativo	Repositorio funcional, estructura organizada, commits ≥5, uso de ramas, acceso compartido verificado	Repositorio creado, estructura básica, commits ≥2	Repositorio incompleto, sin estructura clara o pocos commits (<2)	No presenta repositorio o no funciona
Calidad de documentación: estructura, redacción técnica, coherencia y uso de formato	Documentos claros, coherentes, sin errores, uso adecuado de formato técnico	Documentación comprensible, con mínimos errores	Documentación confusa, con errores frecuentes	Documentación ausente o incorrecta
Evidencia de problema complejo: identificación de ambigüedades, justificación de supuestos, reconocimiento de restricciones y argumentación de decisiones	Evidencia completa, argumenta decisiones con base en restricciones y supuestos, pensamiento crítico evidente	Evidencia básica, identifica y justifica parcialmente	Evidencia limitada, sin argumentación sólida	No evidencia comprensión del problema complejo